

BOOK 5 — WRAPPER CLASSES, GENERICS & COLLECTIONS

Complete Fresher / Entry-Level Java Full Stack Preparation Series

CHAPTER 1 — WRAPPER CLASSES & GENERICS

1.1 Wrapper Classes Definition

A **Wrapper Class** in Java is a class whose object wraps or encloses a primitive data type. It encapsulates a primitive value inside a full Java object allocated on the Heap memory.

- **Package:** `java.lang` (automatically imported).
- **Need for Wrapper Classes:**
 - 1. Collections Framework:** Java Collections (`ArrayList`, `HashSet`, `HashMap`) store ONLY objects, not primitives (`ArrayList<int>` is invalid; `ArrayList<Integer>` is required).
 - 2. Null Values:** Primitives cannot hold `null`, whereas Wrapper objects can represent uninitialized or missing database states (`null`).
 - 3. Utility Methods:** Wrapper classes provide utility functions for type conversions (e.g., `Integer.parseInt()`, `Double.parseDouble()`, `Character.isDigit()`).
 - 4. Data Structure Integration:** Multi-threaded synchronization and reflection APIs operate exclusively on objects.

1.2 Primitive-to-Wrapper Mapping Table

Java provides exactly **8 Wrapper Classes** corresponding to the 8 primitive data types:

Primitive Type	Byte Size	Wrapper Class	Superclass	Default Object Value
byte	1 byte	<code>java.lang.Byte</code>	Number	<code>null</code>
short	2 bytes	<code>java.lang.Short</code>	Number	<code>null</code>
int	4 bytes	<code>java.lang.Integer</code>	Number	<code>null</code>
long	8 bytes	<code>java.lang.Long</code>	Number	<code>null</code>

float	4 bytes	java.lang.Float	Number	null
double	8 bytes	java.lang.Double	Number	null
char	2 bytes	java.lang.Character	Object	null
boolean	1 bit (varies)	java.lang.Boolean	Object	null

1.3 Autoboxing

Autoboxing is the automatic conversion performed by the Java compiler to convert a primitive data type into its corresponding Wrapper object.

- **Compiler Mechanism:** `Integer obj = 100;` is internally transformed by the compiler into `Integer obj = Integer.valueOf(100);`.
- **Example Usage:** Passing a primitive `int` into a method expecting an `Integer` object, or adding a primitive into an `ArrayList<Integer>`.

1.4 Unboxing

Unboxing is the automatic conversion performed by the Java compiler to convert a Wrapper object back into its corresponding primitive data type.

- **Compiler Mechanism:** `int num = obj;` is internally transformed by the compiler into `int num = obj.intValue();`.
- **Example Usage:** Performing arithmetic operations (+, -, *) directly on Wrapper objects triggers implicit unboxing.
- **Common Mistake:** Unboxing a `null` wrapper reference causes a `NullPointerException` at runtime.

1.5 Generics & Type Safety

Generics (introduced in Java 5) allow classes, interfaces, and methods to operate on parameterized types. They provide compile-time type safety and eliminate the need for manual type casting.

- **Need for Generics:** Prior to Java 5, collections stored raw `Object` references. Retrieving elements required explicit downcasting, risking runtime `ClassCastException` if incorrect types were inserted.
- **Benefits:**
 1. **Compile-Time Type Checking:** Detects type mismatch errors during compilation rather than failing at runtime.
 2. **Elimination of Explicit Casting:** No manual `(String)` or `(Integer)` casts required when fetching items.
 3. **Code Reusability:** Write a single generic class/algorithm that works across multiple object types.

- **Type Erasure:** The JVM does NOT retain generic type parameters at runtime. During compilation, the compiler replaces all generic type parameters (T) with Object (or their upper bound) and inserts appropriate casts. This ensures backward compatibility with older Java versions.

1.6 Generic Class

A **Generic Class** is defined with a type parameter in angle brackets <T>:

```
// Generic Class definition
class Box<T> {
    private T item;

    public void setItem(T item) {
        this.item = item;
    }

    public T getItem() {
        return item;
    }
}
```

1.7 Generic Method

A **Generic Method** declares its own type parameters before the return type, allowing it to accept parameters of different generic types independently of the enclosing class:

```
public class GenericMethodDemo {
    // Generic method printing arrays of any object type
    public static <E> void printArray(E[] elements) {
        for (E element : elements) {
            System.out.print(element + " ");
        }
        System.out.println();
    }
}
```

1.8 Wrapper Classes & Generics Coding Programs (Syllabus 21.9.1 to 21.9.7)

Problem 1: Autoboxing Program (21.9.1)

```
public class AutoboxingDemo {
    public static void main(String[] args) {
        int primitiveNum = 25;

        // Autoboxing: Compiler converts primitive int to Integer object via Integer.valueOf()
        Integer wrapperObj = primitiveNum;

        System.out.println("Primitive Value: " + primitiveNum);
        System.out.println("Autoboxed Wrapper Object: " + wrapperObj);
        System.out.println("Class Name: " + wrapperObj.getClass().getName());
    }
}
```

• **Expected Output:**

```
Primitive Value: 25
Autoboxed Wrapper Object: 25
Class Name: java.lang.Integer
```

- **Execution Explanation:** Assigning `primitiveNum` directly to `wrapperObj` triggers compiler autoboxing, converting the primitive stack value into a Heap-allocated `Integer` instance.

Problem 2: Unboxing Program (21.9.2)

```
public class UnboxingDemo {
    public static void main(String[] args) {
        Integer wrapperObj = Integer.valueOf(150);

        // Unboxing: Compiler converts Integer object to primitive int via intValue()
        int primitiveNum = wrapperObj;

        System.out.println("Wrapper Object: " + wrapperObj);
        System.out.println("Unboxed Primitive: " + primitiveNum);
    }
}
```

• **Expected Output:**

```
Wrapper Object: 150
Unboxed Primitive: 150
```

- **Execution Explanation:** The JVM invokes `wrapperObj.intValue()` behind the scenes to extract the primitive `int` value from the object wrapper.

Problem 3: String to Integer Conversion (21.9.3)

```
public class StringToIntegerDemo {
    public static void main(String[] args) {
        String strVal = "456";

        // Converting String to primitive int using Integer.parseInt()
        int parsedInt = Integer.parseInt(strVal);

        // Converting String to Integer object using Integer.valueOf()
        Integer wrapperInt = Integer.valueOf(strVal);

        System.out.println("Parsed Primitive Int: " + (parsedInt + 10)); // 466
        System.out.println("Wrapper Integer Object: " + (wrapperInt + 10)); // 466
    }
}
```

• **Expected Output:**

```
Parsed Primitive Int: 466
Wrapper Integer Object: 466
```

- **Execution Explanation:** `Integer.parseInt()` parses string digits and returns a primitive `int`, whereas `Integer.valueOf()` parses string digits and returns an `Integer` object reference. Passing non-numeric characters (e.g. "456a") throws a `NumberFormatException`.

Problem 4: String to Double Conversion (21.9.4)

```
public class StringToDoubleDemo {
    public static void main(String[] args) {
        String strDouble = "89.75";

        // Converting String to primitive double
        double parsedDouble = Double.parseDouble(strDouble);

        // Converting String to Double object
        Double wrapperDouble = Double.valueOf(strDouble);

        System.out.println("Parsed Primitive Double: " + parsedDouble);
        System.out.println("Wrapper Double Object: " + wrapperDouble);
    }
}
```

• **Expected Output:**

```
Parsed Primitive Double: 89.75
Wrapper Double Object: 89.75
```

- **Execution Explanation:** `Double.parseDouble()` converts string representations of floating-point numbers into primitive double variables.

Problem 5: Generic Class Program (21.9.5)

```
// Generic Repository class
class Repository<T> {
    private T data;

    public Repository(T data) {
        this.data = data;
    }

    public T getData() {
        return data;
    }
}

public class GenericClassDemo {
    public static void main(String[] args) {
        // Instantiating Repository with String type parameter
        Repository<String> stringRepo = new Repository<>("Java Full Stack");
        System.out.println("String Data: " + stringRepo.getData());

        // Instantiating Repository with Integer type parameter
        Repository<Integer> intRepo = new Repository<>(2026);
        System.out.println("Integer Data: " + intRepo.getData());
    }
}
```

- **Expected Output:**

```
String Data: Java Full Stack
Integer Data: 2026
```

- **Execution Explanation:** The diamond operator <> allows Repository to store and return String or Integer objects with strict compile-time type safety without duplicating class logic.

Problem 6: Generic Method Program (21.9.6)

```

public class GenericMethodDemo {
    // Generic display method enforcing type safety across arrays
    public static <T> void displayArray(T[] array) {
        for (T item : array) {
            System.out.print(item + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        String[] strArray = {"Java", "Python", "C++"};
        Integer[] intArray = {10, 20, 30, 40};

        System.out.print("String Array: ");
        displayArray(strArray);

        System.out.print("Integer Array: ");
        displayArray(intArray);
    }
}

```

• **Expected Output:**

```

String Array: Java Python C++
Integer Array: 10 20 30 40

```

- **Execution Explanation:** The generic method `displayArray()` accepts any array of object types (`T[]`), printing elements dynamically without raw type casting.

Problem 7: Generic Collection Program (21.9.7)

```

import java.util.ArrayList;
import java.util.List;

public class GenericCollectionDemo {
    public static void main(String[] args) {
        // Generic List restricted to String objects only
        List<String> frameworkList = new ArrayList<>();

        frameworkList.add("Spring Boot");
        frameworkList.add("Hibernate");
        frameworkList.add("React");
        // frameworkList.add(100); // Compilation Error: Type safety prevents wrong type insert
ion!

        for (String framework : frameworkList) {
            // No explicit downcasting (String) needed due to Generics!
            System.out.println("Framework: " + framework);
        }
    }
}

```

• **Expected Output:**

Framework: Spring Boot
Framework: Hibernate
Framework: React

- **Execution Explanation:** Declaring `List<String>` ensures that only `String` instances can be added to the collection, guaranteeing compile-time type safety and eliminating runtime cast errors.

CHAPTER 2 — JAVA COLLECTIONS FRAMEWORK: LIST, SET, QUEUE & ITERATION

2.1 Collections Introduction

The **Java Collections Framework (JCF)** is a unified architecture in the `java.util` package that provides a set of interfaces, implementation classes, and algorithms to store, manipulate, and process groups of objects efficiently.

2.2 Problems with Arrays

While arrays are fundamental data structures in Java, they have major limitations in real-world application development:

1. **Fixed Size:** Array size must be specified during instantiation and cannot grow or shrink dynamically at runtime.
2. **Manual Element Shifting:** Inserting or deleting elements at arbitrary positions requires writing custom looping code to shift elements.
3. **Lack of Built-in Data Structures:** Arrays do not provide out-of-the-box support for Sets (uniqueness), Queues (FIFO), or Maps (Key-Value pairs).
4. **Limited Built-in Algorithms:** Searching, sorting, and reversing require manual algorithmic implementations or helper class calls.

2.3 Dynamic Collections

Dynamic collections solve array limitations by managing underlying memory automatically:

- **Auto-Resizing:** Collections automatically expand their internal capacity when full and shrink when elements are cleared.

- **Rich Utility API:** Provide pre-built methods (`add()`, `remove()`, `contains()`, `sort()`, `shuffle()`).
- **Standardized Interfaces:** Allow switching underlying implementations (e.g., changing from `ArrayList` to `LinkedList`) with minimal code changes.

2.4 Heterogeneous Data

By default, Java collections store `Object` references. Without Generics (raw types like `List list = new ArrayList()`), collections can store heterogeneous objects (Strings, Integers, custom model instances) within a single container. However, modern Java practice enforces Generics (`List<String>`) for type safety.

2.5 Collection Hierarchy (Basic Structure)

The Java Collections Framework consists of two primary root interface hierarchies:

1. `java.lang.Iterable<E>` Hierarchy (Single-element containers):

- **`Collection<E>` (Interface):** The root interface for lists, sets, and queues.
- **`List<E>` (Ordered, allows duplicates, index-accessible):** Implemented by `ArrayList`, `LinkedList`, `Vector`, `Stack`.
- **`Set<E>` (Unordered/Sorted, unique elements only):** Implemented by `HashSet`, `LinkedHashSet`, `TreeSet`.
- **`Queue<E>` (Processing queues, FIFO/Priority):** Implemented by `PriorityQueue`, `ArrayDeque`, `LinkedList`.

2. `java.util.Map<K, V>` Hierarchy (Key-Value pairs): `Map` is a separate root interface (covered in Chapter 3).

2.6 ArrayList

`java.util.ArrayList<E>` is a resizable-array implementation of the `List` interface. It maintains an ordered sequence of elements and permits duplicate elements and `null` values.

2.7 ArrayList Core Methods

- `add(E e)`: Appends element to the end ($O(1)$ amortized).
- `add(int index, E element)`: Inserts element at specific index ($O(N)$ due to shifting).
- `get(int index)`: Retrieves element at index ($O(1)$ fast random access).
- `set(int index, E element)`: Replaces element at index ($O(1)$).
- `remove(int index)` / `remove(Object o)`: Removes element ($O(N)$ due to shifting).

- `size()`: Returns total element count.
- `contains(Object o)`: Checks if element exists ($O(N)$ linear search).
- `clear()`: Removes all elements.

2.8 ArrayList Traversal

ArrayList elements can be traversed using 5 primary techniques:

1. Standard for loop with index (`list.get(i)`).
2. Enhanced for-each loop (`for (T item : list)`).
3. `Iterator<E>` (`iterator.hasNext()`, `iterator.next()`).
4. `ListIterator<E>` (allows bi-directional traversal: `hasPrevious()`, `previous()`).
5. `forEach()` method with Lambda expression (`list.forEach(item -> ...)`).

2.9 ArrayList Internal Array Mechanics

Internally, ArrayList uses an `Object[] elementData` array buffer to store elements:

```
transient Object[] elementData; // Internal buffer storing elements
```

2.10 Capacity & Resizing Mechanism

- **Default Initial Capacity:** When an ArrayList is initialized via `new ArrayList<>()`, an empty array buffer is created. Upon adding the first element, its default initial capacity becomes **10**.
- **Resizing Mechanics:** ArrayList growth is implementation-dependent. In common OpenJDK implementations, capacity is typically increased by approximately 50% when resizing is required.

2.11 Element Shifting ($O(N)$ Insertion/Deletion)

When inserting or removing elements at an arbitrary index inside an ArrayList, Java executes `System.arraycopy()` to shift all subsequent elements left or right by one position. Consequently, arbitrary insertions and deletions take $O(N)$ linear time.

2.12 LinkedList

`java.util.LinkedList<E>` is a doubly-linked list implementation of the `List` and `Deque` interfaces.

2.13 LinkedList Nodes

LinkedList does NOT use an underlying array buffer. Instead, it maintains elements in separate heap-allocated Node objects linked sequentially:

```
private static class Node<E> {  
    E item;           // Stored data element  
    Node<E> next;     // Pointer to next Node  
    Node<E> prev;     // Pointer to previous Node  
}
```

2.14 LinkedList Insertion & Deletion ($O(1)$ at Ends)

- **Fast End Operations:** Inserting or deleting nodes at the head or tail (`addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`) takes $O(1)$ constant time because only pointers are updated.
- **Slow Random Access:** Accessing elements by index (`get(index)`) takes $O(N)$ linear time because LinkedList must traverse nodes sequentially from the head or tail.

2.15 Set Interface

`java.util.Set<E>` represents an unordered collection that contains **NO duplicate elements** (models the mathematical set abstraction).

2.16 HashSet

- **Underlying Structure:** Backed by an internal `java.util.HashMap` instance.
- **Unordered:** Does NOT guarantee any specific iteration order of elements.
- **Performance:** `HashSet` `add()`, `remove()`, and `contains()` are expected to be $O(1)$ on average with good hash distribution; worst-case performance can be worse.
- **Null Support:** Allows a single `null` element.

2.17 LinkedHashSet

- **Underlying Structure:** Extends `HashSet` and is backed by a `HashMap` + doubly-linked list running through all entries.
- **Insertion-Ordered:** Preserves the exact **insertion order** of elements during iteration.
- **Performance:** Average expected time is $O(1)$ for basic operations, maintaining predictable iteration order via its doubly-linked list.

2.18 TreeSet

- **Underlying Structure:** Implements `NavigableSet` and is backed by a Red-Black self-balancing binary search tree (`TreeMap`).
- **Sorted Order:** Elements are stored in **sorted order** (either according to their natural ordering via `Comparable`, or via a custom `Comparator`).
- **Performance:** Basic operations (`add`, `remove`, `contains`) take $O(\log N)$ logarithmic time.
- **Null Handling:** With natural ordering, `null` generally cannot be compared with other elements. A custom `Comparator` may handle `null` differently if it explicitly supports `null` comparison.

2.19 Queue Interface

`java.util.Queue<E>` is a collection designed for holding elements prior to processing, adhering strictly to First-In-First-Out (**FIFO**) ordering.

2.20 Stack

`java.util.Stack<E>` is a legacy class extending `Vector` that models a Last-In-First-Out (**LIFO**) stack of objects.

2.21 LIFO Stack Mechanics

- `push(E item)`: Pushes an item onto the top of the stack.
- `pop()`: Removes and returns the object at the top of the stack.
- `peek()`: Looks at the object at the top of the stack without removing it.

2.22 ArrayDeque

`java.util.ArrayDeque<E>` is a resizable-array implementation of the `Deque` (Double-Ended Queue) interface.

- **Superior Replacement:** Modern Java applications prefer `ArrayDeque` over `Stack` (for LIFO stacks) and `LinkedList` (for FIFO queues).
- **Performance:** Faster than `Stack` and `LinkedList` because it avoids node allocation overhead and pointer chasing. Does NOT permit `null` elements.

2.23 PriorityQueue

`java.util.PriorityQueue<E>` is an unbounded, heap-based priority queue:

- **Heap-Based Structure:** Implemented internally using a binary heap data structure.
- **Highest Priority Element:** Natural ordering gives the smallest element the highest priority at the head of the queue.
- **Retrieval via `poll()`:** `poll()` removes and returns the highest-priority element.
- **Iteration Behavior:** Iterating over a `PriorityQueue` (via `for-each` or `Iterator`) does **NOT** guarantee sorted order.
- **Custom Ordering:** A custom `Comparator` passed to the constructor can define another priority ordering.

2.24 Natural Ordering

Natural ordering refers to the default comparison logic defined by a class implementing the `java.lang.Comparable` interface (e.g., numerical order for `Integer`, alphabetical order for `String`).

2.25 Iterator Interface

`java.util.Iterator<E>` is an object that allows sequential traversal through a collection:

- `hasNext()`: Returns `true` if the iteration has more elements.
- `next()`: Returns the next element in the iteration.
- `remove()`: Removes the last element returned by `next()` from the underlying collection safely.

2.26 Enhanced for-each Loop Traversal

The enhanced `for-each` loop (`for (T item : collection)`) is syntactic sugar compiled down to an `Iterator` call for any class implementing `Iterable`.

2.27 Structural Modification

Structural modification refers to any operation that adds, deletes, or clears elements from a collection, altering its size or internal bucket structure.

2.28 ConcurrentModificationException

`ConcurrentModificationException` is thrown when a thread or loop attempts to structurally modify a collection while iterating over it using a traditional loop or `Iterator`, without using the `Iterator`'s own `remove()` method.

2.29 Fail-Fast Iterators

- **Mechanics:** Iterators for standard collections (`ArrayList`, `HashSet`, `HashMap`) are **Fail-Fast**. They maintain an internal modification count variable (`modCount`).
- **Detection:** During every `next()` call, the iterator compares `modCount` with its expected value (`expectedModCount`). If a mismatch is detected (indicating the collection was modified externally), the iterator immediately throws `ConcurrentModificationException`.

2.30 CopyOnWriteArrayList (Snapshot Iteration)

`java.util.concurrent.CopyOnWriteArrayList<E>` is a thread-safe variant of `ArrayList`:

- **Mechanics:** Every mutative operation (`add`, `set`, `remove`) makes a fresh, underlying copy of the entire internal array buffer.
- **Snapshot Iteration:** Its iterator operates on an immutable snapshot of the underlying array taken at the time the iterator was created. Therefore, subsequent structural modifications to the list do not cause `ConcurrentModificationException` for that iterator. (Note: snapshot iteration is the mechanism used here; 'fail-safe' is not an official Java Collections category).

CHAPTER 3 — HASHING, MAPS, SORTING & LEGACY COLLECTIONS

3.1 Hashing Fundamentals

Hashing is the process of mapping a key or object into a 32-bit signed integer hash value using a hash function. In Java, hashing powers high-performance data structures like `HashSet`, `HashMap`, `LinkedHashMap`, and `Hashtable`.

3.2 Hash Function

A **Hash Function** is an algorithm that accepts an input key and produces a fixed-size integer hash code. A good hash function distributes keys uniformly across table buckets to minimize collisions.

3.3 hashCode() Role

The public `int hashCode()` method inherited from `java.lang.Object` calculates the integer hash value of an object. Hash-based collections call `key.hashCode()` to compute bucket indices.

3.4 Hash Table & Buckets Architecture

Internally, a `HashMap` maintains an array of buckets (`Node<K, V>[] table`):

- **Bucket Index Calculation:** The bucket index for a key is calculated using bitwise masking:

$$\text{Bucket Index} = \text{hash} \ \& \ (\text{capacity} - 1)$$

- **Node Structure:** Each bucket index holds a reference to a `Node<K, V>` containing `int hash`, `K key`, `V value`, and `Node<K, V> next`.

3.5 Collision Handling (Separate Chaining & Treeification)

A **Collision** occurs when two distinct keys yield identical bucket indices.

- **Separate Chaining (Linked List):** `HashMap` handles collisions by storing multiple key-value pairs at the same bucket index as a singly-linked list of nodes.
- **Treeification (Java 8 Optimization):** If a single bucket's linked list grows beyond **8 elements** (and total map capacity is at least **64**), the linked list converts into a **Red-Black Tree** (`TreeNode`). Note that thresholds 8 and 64 are common OpenJDK implementation details, NOT Java API guarantees.

3.6 Load Factor & Resizing Threshold

- **Default Load Factor:** `0.75` (75%).
- **Threshold Calculation:** $\text{Threshold} = \text{Capacity} \times \text{Load Factor}$.
- **Resizing Mechanics:** `HashMap` resizing behavior is implementation-dependent. In common OpenJDK implementations, capacity is typically doubled during resizing.

3.7 Sorting Collections

Sorting rearranges collection elements into a specific ascending or descending order.

3.8 Collections.sort()

`java.util.Collections.sort(List<T> list)` sorts the specified list into ascending order according to the natural ordering of its elements. Internally, Java uses Timsort (an adaptive, stable merge-sort algorithm with

$O(N \log N)$ complexity).

3.9 Comparable Interface (Natural Ordering)

`java.lang.Comparable<T>` is implemented inside a domain model class (e.g., `Student`) to define its single **natural ordering**:

```
public interface Comparable<T> {  
    public int compareTo(T o);  
}
```

- **Return Values:**
- **Negative int (< 0):** Current object `this` is smaller than object `o`.
- **Zero (== 0):** Current object `this` is equal to object `o`.
- **Positive int (> 0):** Current object `this` is greater than object `o`.

3.10 Comparator Interface (Custom Ordering)

`java.util.Comparator<T>` is a functional interface used to define **multiple custom sorting strategies** outside the model class:

```
@FunctionalInterface  
public interface Comparator<T> {  
    public int compare(T o1, T o2);  
}
```

3.11 Sorting Objects

Sorting custom domain objects (e.g., sorting `Employee` objects by salary or name) requires passing a `Comparator` or ensuring the class implements `Comparable`.

3.12 Map Interface

`java.util.Map<K, V>` represents a data structure that maps **unique keys to values**. Maps do NOT inherit from `java.util.Collection`.

3.13 Map Interface Core Methods

- `put(K key, V value)`: Inserts key-value mapping (replaces old value if key exists).
- `get(Object key)`: Returns value associated with key (or `null` if absent).
- `remove(Object key)`: Removes mapping for specified key.
- `containsKey(Object key)`: Checks if key exists ($O(1)$).
- `containsValue(Object value)`: Checks if value exists ($O(N)$ linear scan).
- `size()`: Returns total key-value mappings.

3.14 HashMap

- **Unordered**: Does NOT preserve key insertion order.
- **Null Support**: Allows **one** `null` **key** and **multiple** `null` **values**.
- **Performance**: `HashMap` `get()`, `put()`, and `remove()` are expected to be $O(1)$ on average with good hash distribution; worst-case performance can be worse.

3.15 LinkedHashMap

- Extends `HashMap` and preserves **key insertion order** (or access order) using a doubly-linked list across all buckets.

3.16 TreeMap

`java.util.TreeMap<K, V>` is a `NavigableMap` backed by a Red-Black tree.

- Keys are stored in **sorted order**.
- **Null Key Handling**: With natural ordering, `null` keys generally cannot be compared. A custom `Comparator` may explicitly handle `null`. Search complexity is $O(\log N)$.

3.17 keySet()

Returns a `Set<K>` containing all unique keys present in the map.

3.18 values()

Returns a `Collection<V>` containing all values present in the map (may contain duplicates).

3.19 entrySet()

Returns a `Set<Map.Entry<K,V>>` containing key-value pair objects (`Map.Entry`).

3.20 Map Traversal Techniques

1. Traversing via `entrySet()` (Most efficient for accessing both Key and Value):

```
for (Map.Entry<String, Integer> entry : map.entrySet()) {  
    System.out.println(entry.getKey() + " -> " + entry.getValue());  
}
```

2. Traversing via `keySet()` and `map.get(key)`.

3. Java 8 `forEach()` Lambda: `map.forEach((k, v) -> System.out.println(k + " -> " + v));`.

3.21 Legacy Collections

Legacy collections are synchronized classes introduced in Java 1.0 before the Java Collections Framework was established in Java 1.2.

3.22 Dictionary

`java.util.Dictionary<K,V>` is an obsolete abstract superclass replaced by the `Map` interface.

3.23 Properties

`java.util.Properties` extends `Hashtable`. It stores key-value pairs where both keys and values are `String` objects. Commonly used to read `.properties` application configuration files.

3.24 Vector

`java.util.Vector<E>` is a thread-safe, resizable array. All its methods are synchronized. Its growth strategy is implementation-dependent; common implementations may double capacity when no increment is specified.

3.25 Hashtable

`java.util.Hashtable<K, V>` is a thread-safe hash table implementation. All methods are synchronized. Does NOT allow null keys or null values.

CHAPTER 4 — COMPLETE COLLECTIONS & GENERICS CODING PRACTICE

Common Collections Coding Problems (Syllabus 22.56.1 to 22.56.21)

Problem 1: Add / Remove / Search in ArrayList (22.56.1)

```
import java.util.ArrayList;
import java.util.List;

public class ArrayListBasicDemo {
    public static void main(String[] args) {
        List<String> cities = new ArrayList<>();

        // Adding elements
        cities.add("Mumbai");
        cities.add("Delhi");
        cities.add("Bangalore");
        cities.add("Chennai");

        // Searching element
        System.out.println("Contains Bangalore? " + cities.contains("Bangalore"));

        // Removing element by name and by index
        cities.remove("Delhi");
        cities.remove(0); // Removes Mumbai

        System.out.println("Remaining Cities: " + cities);
    }
}
```

• Expected Output:

```
Contains Bangalore? true
Remaining Cities: [Bangalore, Chennai]
```

Problem 2: Iterate ArrayList (22.56.2)

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IterateArrayListDemo {
    public static void main(String[] args) {
        List<String> list = List.of("Java", "Spring", "Microservices");

        // Using Iterator
        System.out.print("Iterator Traversal: ");
        Iterator<String> it = list.iterator();
        while (it.hasNext()) {
            System.out.print(it.next() + " ");
        }
        System.out.println();
    }
}
```

• **Expected Output:**

```
Iterator Traversal: Java Spring Microservices
```

Problem 3: Remove Duplicates Using HashSet (22.56.3)

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class RemoveDuplicatesDemo {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(10, 20, 10, 30, 20, 40);

        // Passing collection to HashSet automatically strips duplicates
        Set<Integer> uniqueNumbers = new HashSet<>(numbers);

        System.out.println("Original List: " + numbers);
        System.out.println("Unique Set: " + uniqueNumbers);
    }
}
```

• **Expected Output:**

```
Original List: [10, 20, 10, 30, 20, 40]
Unique Set: [20, 40, 10, 30]
```

Problem 4: Find Duplicate Elements Using HashSet (22.56.4)

```
import java.util.HashSet;
import java.util.Set;

public class FindDuplicatesDemo {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 2, 5, 3, 6};
        Set<Integer> set = new HashSet<>();
        Set<Integer> duplicates = new HashSet<>();

        for (int num : arr) {
            // set.add() returns false if element already exists
            if (!set.add(num)) {
                duplicates.add(num);
            }
        }
        System.out.println("Duplicate Elements: " + duplicates);
    }
}
```

• **Expected Output:**

```
Duplicate Elements: [2, 3]
```

Problem 5: Maintain Insertion Order Using LinkedHashSet (22.56.5)

```
import java.util.LinkedHashSet;
import java.util.Set;

public class LinkedHashSetDemo {
    public static void main(String[] args) {
        Set<String> set = new LinkedHashSet<>();
        set.add("First");
        set.add("Second");
        set.add("Third");
        set.add("First"); // Duplicate ignored

        System.out.println("LinkedHashSet Elements (Preserves Order): " + set);
    }
}
```

• **Expected Output:**

```
LinkedHashSet Elements (Preserves Order): [First, Second, Third]
```

Problem 6: Sort Elements Using TreeSet (22.56.6)

```
import java.util.Set;
import java.util.TreeSet;

public class TreeSetDemo {
    public static void main(String[] args) {
        Set<Integer> numbers = new TreeSet<>();
        numbers.add(45);
        numbers.add(12);
        numbers.add(89);
        numbers.add(23);

        System.out.println("TreeSet Sorted Elements: " + numbers);
    }
}
```

• **Expected Output:**

```
TreeSet Sorted Elements: [12, 23, 45, 89]
```

Problem 7: Reverse a List (22.56.7)

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class ReverseListDemo {
    public static void main(String[] args) {
        List<String> items = new ArrayList<>(List.of("A", "B", "C", "D"));

        Collections.reverse(items);

        System.out.println("Reversed List: " + items);
    }
}
```

• **Expected Output:**

```
Reversed List: [D, C, B, A]
```

Problem 8: Sort an ArrayList (22.56.8)

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class SortArrayListDemo {
    public static void main(String[] args) {
        List<Integer> scores = new ArrayList<>(List.of(88, 45, 92, 67));

        Collections.sort(scores);

        System.out.println("Sorted Scores: " + scores);
    }
}
```

• **Expected Output:**

```
Sorted Scores: [45, 67, 88, 92]
```

Problem 9: Sort Objects Using Comparable (22.56.9)

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

class Student implements Comparable<Student> {
    int id;
    String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Sorting by ID in ascending order
    @Override
    public int compareTo(Student o) {
        return Integer.compare(this.id, o.id);
    }

    @Override
    public String toString() {
        return "Student{" + "id=" + id + ", name='" + name + '\'' + '}';
    }
}

public class ComparableDemo {
    public static void main(String[] args) {
        List<Student> students = new ArrayList<>();
        students.add(new Student(103, "Rohan"));
        students.add(new Student(101, "Aarav"));
        students.add(new Student(102, "Priya"));

        Collections.sort(students);

        System.out.println("Sorted Students by ID: " + students);
    }
}
```

• **Expected Output:**

```
Sorted Students by ID: [Student{id=101, name='Aarav'}, Student{id=102, name='Priya'}, Student{id=103, name='Rohan'}]
```

Problem 10: Sort Objects Using Comparator (22.56.10)

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

class Employee {
    int id;
    String name;
    double salary;

    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    @Override
    public String toString() {
        return name + ":" + salary;
    }
}

public class ComparatorDemo {
    public static void main(String[] args) {
        List<Employee> list = new ArrayList<>();
        list.add(new Employee(1, "Suresh", 65000));
        list.add(new Employee(2, "Anil", 85000));
        list.add(new Employee(3, "Meena", 52000));

        // Sorting by Salary descending using Comparator lambda
        Collections.sort(list, (e1, e2) -> Double.compare(e2.salary, e1.salary));

        System.out.println("Employees Sorted by Salary (Descending): " + list);
    }
}
```

• Expected Output:

```
Employees Sorted by Salary (Descending): [Anil:85000.0, Suresh:65000.0, Meena:52000.0]
```

Problem 11: Count Frequency Using HashMap (22.56.11)

```
import java.util.HashMap;
import java.util.Map;

public class FrequencyCountDemo {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 2, 3, 3, 3, 4};
        Map<Integer, Integer> freqMap = new HashMap<>();

        for (int num : numbers) {
            freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);
        }

        System.out.println("Element Frequencies: " + freqMap);
    }
}
```

• **Expected Output:**

```
Element Frequencies: {1=1, 2=2, 3=3, 4=1}
```

Problem 12: Find Duplicate Values Using HashMap (22.56.12)

```
import java.util.HashMap;
import java.util.Map;

public class FindDuplicateValuesDemo {
    public static void main(String[] args) {
        Map<String, String> map = new HashMap<>();
        map.put("E1", "Java");
        map.put("E2", "Python");
        map.put("E3", "Java");

        Map<String, Integer> valCount = new HashMap<>();
        for (String val : map.values()) {
            valCount.put(val, valCount.getOrDefault(val, 0) + 1);
        }

        System.out.println("Value Frequencies: " + valCount);
    }
}
```

• **Expected Output:**

```
Value Frequencies: {Java=2, Python=1}
```

Problem 13: Character Frequency Using HashMap (22.56.13)

```
import java.util.HashMap;
import java.util.Map;

public class CharFrequencyDemo {
    public static void main(String[] args) {
        String str = "collections";
        Map<Character, Integer> charMap = new HashMap<>();

        for (char ch : str.toCharArray()) {
            charMap.put(ch, charMap.getOrDefault(ch, 0) + 1);
        }

        System.out.println("Character Frequencies: " + charMap);
    }
}
```

• **Expected Output:**

```
Character Frequencies: {c=2, s=1, e=1, t=1, i=2, l=2, o=2, n=1}
```

Problem 14: Iterate HashMap (22.56.14)

```
import java.util.HashMap;
import java.util.Map;

public class IterateHashMapDemo {
    public static void main(String[] args) {
        Map<Integer, String> map = Map.of(101, "Amit", 102, "Neha");

        System.out.println("Iterating Map EntrySet:");
        for (Map.Entry<Integer, String> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " => " + entry.getValue());
        }
    }
}
```

• **Expected Output:**

```
Iterating Map EntrySet:
101 => Amit
102 => Neha
```

Problem 15: Sort Map by Keys (22.56.15)

```
import java.util.HashMap;
import java.util.Map;
import java.util.TreeMap;

public class SortMapByKeysDemo {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("Banana", 30);
        map.put("Apple", 50);
        map.put("Cherry", 20);

        // Passing HashMap to TreeMap sorts keys naturally
        Map<String, Integer> sortedMap = new TreeMap<>(map);

        System.out.println("Sorted Map by Keys: " + sortedMap);
    }
}
```

• **Expected Output:**

```
Sorted Map by Keys: {Apple=50, Banana=30, Cherry=20}
```

Problem 16: Sort Map by Values (22.56.16)

```
import java.util.*;

public class SortMapByValuesDemo {
    public static void main(String[] args) {
        Map<String, Integer> map = Map.of("A", 40, "B", 10, "C", 30);

        List<Map.Entry<String, Integer>> entryList = new ArrayList<>(map.entrySet());
        entryList.sort(Map.Entry.comparingByValue());

        System.out.println("Sorted Entries by Value:");
        for (Map.Entry<String, Integer> entry : entryList) {
            System.out.println(entry.getKey() + " = " + entry.getValue());
        }
    }
}
```

• **Expected Output:**

```
Sorted Entries by Value:
B = 10
C = 30
A = 40
```

Problem 17: Find Maximum / Minimum in Collection (22.56.17)

```
import java.util.Collections;
import java.util.List;

public class MinMaxCollectionDemo {
    public static void main(String[] args) {
        List<Integer> list = List.of(34, 12, 89, 5, 67);

        int min = Collections.min(list);
        int max = Collections.max(list);

        System.out.println("Minimum Value: " + min);
        System.out.println("Maximum Value: " + max);
    }
}
```

• **Expected Output:**

```
Minimum Value: 5
Maximum Value: 89
```

Problem 18: Remove Elements Using Iterator (22.56.18)

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IteratorRemoveDemo {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>(List.of(10, 15, 20, 25, 30));

        Iterator<Integer> it = numbers.iterator();
        while (it.hasNext()) {
            if (it.next() % 2 != 0) {
                // Safe removal using Iterator to prevent ConcurrentModificationException
                it.remove();
            }
        }
        System.out.println("Even Numbers List: " + numbers);
    }
}
```

• **Expected Output:**

```
Even Numbers List: [10, 20, 30]
```

Problem 19: Stack Push / Pop (22.56.19)

```
import java.util.Stack;

public class StackDemo {
    public static void main(String[] args) {
        Stack<String> stack = new Stack<>();

        stack.push("Page 1");
        stack.push("Page 2");
        stack.push("Page 3");

        System.out.println("Top Page (peek): " + stack.peek());
        System.out.println("Popped Page: " + stack.pop());
        System.out.println("Remaining Stack: " + stack);
    }
}
```

• **Expected Output:**

```
Top Page (peek): Page 3
Popped Page: Page 3
Remaining Stack: [Page 1, Page 2]
```

Problem 20: Queue Add / Remove (22.56.20)

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueDemo {
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>();

        queue.offer("Customer 1");
        queue.offer("Customer 2");

        System.out.println("Served: " + queue.poll());
        System.out.println("Next in Queue: " + queue.peek());
    }
}
```

• **Expected Output:**

```
Served: Customer 1
Next in Queue: Customer 2
```

Problem 21: PriorityQueue Program (22.56.21)

```
import java.util.PriorityQueue;
import java.util.Queue;

public class PriorityQueueDemo {
    public static void main(String[] args) {
        // Min-Heap PriorityQueue
        Queue<Integer> pq = new PriorityQueue<>();

        pq.offer(40);
        pq.offer(10);
        pq.offer(30);

        System.out.println("Polled Least Element: " + pq.poll()); // 10
        System.out.println("Polled Next Element: " + pq.poll()); // 30
    }
}
```

• **Expected Output:**

```
Polled Least Element: 10
Polled Next Element: 30
```

BOOK 5 END-OF-BOOK REVISION & INTERVIEW PREPARATION PACK

1. COMPLETE BOOK 5 REVISION SHEET

- **Autoboxing / Unboxing:** Automatic primitive $\rightarrow$ wrapper conversion. Unboxing null throws `NullPointerException`.
- **Generics:** Parameterized type safety at compile time; erased to bounds or `Object` at runtime via Type Erasure.
- **ArrayList vs LinkedList:** `ArrayList` uses contiguous dynamic array ($O(1)$ random access, $O(N)$ arbitrary insertion/deletion); `LinkedList` uses doubly-linked nodes ($O(1)$ head/tail insertion/deletion, $O(N)$ lookup).
- **Set Types:** `HashSet` (unordered, $O(1)$), `LinkedHashSet` (insertion order, $O(1)$), `TreeSet` (sorted Red-Black tree, $O(\log N)$, no null).
- **Hashing Architecture:** `Bucket index = hash & (capacity - 1)`. Separate chaining linked lists convert to Red-Black trees when bucket node count exceeds 8.
- **HashMap vs ConcurrentHashMap vs Hashtable:** `HashMap` is non-synchronized; `Hashtable` synchronizes entire methods; `ConcurrentHashMap` uses bucket/segment locks.
- **Comparable vs Comparator:** `Comparable (compareTo)` provides single natural ordering in class; `Comparator (compare)` provides custom external sorting.

2. IMPORTANT FRESHER INTERVIEW QUESTIONS

1. What is the difference between ArrayList and LinkedList?

- ***Answer:** ArrayList is backed by a dynamic array buffer providing $O(1)$ random access but $O(N)$ arbitrary insertions/deletions due to element shifting. LinkedList is a doubly-linked list providing $O(1)$ insertions/deletions at ends but $O(N)$ sequential lookup.

2. How does HashMap handle hash collisions in Java 8?

- ***Answer:** HashMap uses separate chaining (singly-linked list nodes inside bucket indices). In Java 8, if a bucket's node count exceeds 8 (and total capacity ≥ 64), the linked list treeifies into a Red-Black Tree, improving lookup complexity from $O(N)$ to $O(\log N)$.

3. What is ConcurrentModificationException and how do you avoid it?

- ***Answer:** It occurs when a fail-fast iterator detects an external structural modification (modCount mismatch) during collection traversal. Avoid it using `Iterator.remove()`, `CopyOnWriteArrayList`, or `ConcurrentHashMap`.

3. IMPORTANT OUTPUT-PREDICTION QUESTIONS

Question 1: Unboxing Null Wrapper Reference

```
public class NullUnboxingOutput {
    public static void main(String[] args) {
        Integer num = null;
        int primitiveNum = num; // Unboxing
        System.out.println(primitiveNum);
    }
}
```

• Output:

```
Exception in thread "main" java.lang.NullPointerException
```

- **Explanation:** Unboxing invokes `num.intValue()`. Calling a method on a null reference triggers `NullPointerException`.

4. DEBUGGING PRACTICE & COMMON MISTAKES

- **Bug:** Adding elements to a custom object HashSet without overriding `hashCode()` and `equals()`.

- ***Cause:** Default `Object.hashCode()` produces bucket mismatches, causing duplicate custom objects to be inserted into a `HashSet`.
- ***Fix:** Override both `equals()` and `hashCode()` consistently using unique instance fields.

5. FRESHER-LEVEL MOCK INTERVIEW SCENARIO

Interviewer: Can you explain the difference between `Comparable` and `Comparator` in Java?

Candidate: Yes, sir! `Comparable` is an interface in `java.lang` that defines natural ordering for a class by overriding `compareTo(T o)` directly inside the class. `Comparator` is a functional interface in `java.util` that defines custom external sorting logic by implementing `compare(T o1, T o2)`. We use `Comparator` when we want multiple sorting strategies (e.g., sorting `Employees` by Name OR Salary) without modifying the original class.

6. CONCEPT COMPARISON TABLES

Feature	<code>ArrayList</code>	<code>LinkedList</code>	<code>HashSet</code>	<code>TreeSet</code>	<code>HashMap</code>	<code>TreeMap</code>
Data Structure	Dynamic Array	Doubly-Linked List	Hash Table	Red-Black Tree	Hash Table	Red-Black Tree
Duplicates	Allowed	Allowed	Prohibited	Prohibited	Unique Keys	Unique Keys
Null Support	Allowed	Allowed	1 null	No null	1 null Key	No null Key
Time Complexity	$O(1)$ Access	$O(1)$ Ends	$O(1)$ Expected Search	$O(\log N)$	$O(1)$ Expected Search	$O(\log N)$

7. COMPLETE BOOK 5 SYLLABUS COVERAGE CHECKLIST

- [X] **Section 21: Wrapper Classes & Generics (21.1 to 21.9.7)**
- [X] **Section 22: Java Collections Framework (22.1 to 22.56.21)**
- [X] List, Set, Queue, Stack, ArrayDeque, PriorityQueue
- [X] Fail-Fast Iterators & CopyOnWriteArrayList Snapshot Iteration
- [X] Hashing, Buckets, Collision Treeification, Load Factor
- [X] HashMap, LinkedHashMap, TreeMap, Legacy Collections
- [X] Comparable vs Comparator

- [x] All 28 Wrapper, Generics & Collections coding problems solved